Nhill 2.1 — Build and release guide

Français : ../fr/guide-compilation.md

How to produce, verify and assemble the 2.1.0 deliverables on a **fresh** machine. All scripts live in `scripts/`, are written in strict Bash (`set -Eeuo pipefail`) and share `scripts/lib/common.sh`.

1. What is built where

Deliverable	Build machine	Script	Output
Server + 3 applications (Linux), tests	Debian 13 or Ubuntu 24.04 amd64	`build-linux.sh`	`build/`
`.deb` packages	the **target** distribution (one set per distribution)	`build-deb.sh`	`dist/linux/<distribution>/`
Windows installers (NSIS `.exe`)	Windows 10/11 + Visual Studio 2022 + Qt 6.5 MSVC (recommended); or Linux + MinGW + Qt for Windows	`packaging/windows/build-windows.ps1` ; `build-windows.sh`	`build-windows/dist/` ; `dist/windows/`
Android APK (Nhill Business)	Linux + JDK 17 + Android SDK/NDK + Qt for Android	`build-android.sh`	`dist/android/`
Documentation PDFs	any machine with pandoc + a PDF engine	`build-docs-pdf.sh`	`docs/pdf/` , `legal/pdf/` , `marketing/pdf/` , `README*.pdf`
Website (archive)	any machine with Python 3	`build-website.sh`	`dist/website/`
Deliverable check	—	`verify-build.sh`	`build/logs/verify-build-*.log`
Release folder	—	`package-release.sh`	`release/V2.1.0/`
Everything possible here	—	`build-all.sh`	summary succeeded / skipped / failed

A `.deb` package depends on its distribution's library versions: do not copy an Ubuntu package to Debian (or the reverse). The CI builds both sets in clean containers.

2. Common conventions

Option / code	Meaning
`--dry-run`	prints the commands without running or installing anything
`--force`	allows overwriting an existing output folder
`--output DIR`	output folder
`--server-url URL`	server address compiled into the applications (HTTPS in production)
`-h` , `--help`	help

Option / code	Meaning
code 0	success
code 1	failure (build, test, check)
code 2	wrong usage (unknown option)
code 3	incompatible environment (system, tool or SDK missing) — the message says what to install

Logs: `build/logs/<script>-<date>.log`. Each output folder receives a `SHA256SUMS` file. No script contains a secret; signing keys are given through environment variables at build time. Script messages are in French.

3. Linux

3.1 Dependencies (fresh machine)

```
scripts/install-deps-linux.sh --dry-run --all      # see what would be installed
scripts/install-deps-linux.sh --with-packaging --with-docs --yes
```

Only the distribution's official repositories (`apt`). Validated distributions: Debian 13, Ubuntu 24.04 (`--force-distro` to try a derivative, without guarantee). An inventory of the installed versions is written to `build/logs/deps-manifest-*.txt`.

Without access to the `apt` repositories (filtered network): `scripts/bootstrap-deps-from-source.sh` builds the dependencies from precise tags of their official repositories into a private prefix. Fallback only: prefer the distribution packages, which receive security fixes (cpp-httplib 0.14 in particular, see the security audit).

3.2 Build and tests

```
scripts/build-linux.sh                          # everything + tests
scripts/build-linux.sh --server-only --jobs 4
scripts/build-linux.sh --client-only --no-tests --server-url https://api.example.cd
```

3.3 Debian packages

```
scripts/build-deb.sh --server-url https://api.example.cd
# -> dist/linux/ubuntu-24.04/nhill-{server,business,pos,console}_2.1.0-1_amd64.deb
#    (or dist/linux/debian-13/ on Debian 13)
```

Each package's dependencies are computed by `dpkg-shlibdeps`; `lintian` checks the packages. Installation check on a clean machine: `sudo apt install ./dist/linux/<distribution>/nhill-business_2.1.0-1_amd64.deb`.

4. Windows

4.1 On Windows (recommended)

Prerequisites: Visual Studio 2022 ("Desktop development with C++" workload), CMake ≥ 3.21, Ninja, Git, Qt 6.5.3 MSVC 2019 64-bit, NSIS 3. QtKeychain ("Remember me", Windows Credential Manager) does not need to be installed: the script builds it from its official source (tag 0.17.0, verified commit) with `packaging\windows\build-qtkeychain.ps1`, as a **static** library.

```
powershell -ExecutionPolicy Bypass -File packaging\windows\build-windows.ps1 `
  -Source C:\src\nhill -QtDir C:\Qt\6.5.3\msvc2019_64 -ServerUrl https://api.example.cd `
  -BuildDir C:\src\nhill\build-windows
```

`-ExecutionPolicy Bypass` applies to this command only; do not change the machine's execution policy. Result, in `build-windows\dist\`: `nhill-business-2.1.0-setup.exe`, `nhill-pos-2.1.0-setup.exe`, `nhill-console-2.1.0-setup.exe` and `SHA256SUMS`.

QtKeychain on Windows:

Option	Effect
(none)	QtKeychain is built into `<BuildDir>\deps\qtkeychain` on the first build, then reused
`-KeychainDir C:\build\qtkeychain`	uses an existing static build (made by `build-qtkeychain.ps1 -QtDir … -Prefix …`)
`-AllowNoKeychain`	builds without QtKeychain: the session is **never** remembered (no plain-text fallback)

Why a static library: `windeployqt` treats any DLL named `Qt6*.dll` as a Qt module and only looks for it in Qt's folder, so a `qt6keychain.dll` shipped next to the application makes the deployment fail. Linked statically, QtKeychain is part of every `.exe`. After the build, the script checks with `dumpbin` that each application imports `CredReadW`, `CredWriteW` and `CredDeleteW` from `ADVAPI32` and depends on no QtKeychain DLL. The vcpkg QtKeychain package (a DLL) is therefore not used, even with `-VcpkgRoot`.

Real test of the secure storage: `-DNHILL_BUILD_KEYCHAIN_TEST=ON` (set by the script) produces `build-windows\tests\keychain\test_keychain.exe`; deployed with `windeployqt`, it writes a dummy session to Credential Manager, reads it back from a second process, then erases it (`NHILL_KEYCHAIN_REQUIRED=1` turns a missing storage into a failure instead of a skipped test).

4.2 From Linux (cross-compilation)

```
scripts/install-deps-linux.sh --with-windows-cross --yes
packaging/windows/build-qt-mingw.sh               # Qt for Windows (several hours)
scripts/build-windows.sh --qt-prefix /opt/nhill-win/qt6
```

Limit: the MinGW cross-build does not include QtKeychain; applications built this way never remember the session (they stay safe, the option is simply unavailable). To ship "Remember me" on Windows, use method 4.1.

4.3 Signing

Sign the `.exe` files and installers (Authenticode, `signtool sign /fd sha256 /tr <timestamp> …`) with the publisher's certificate before any distribution. Without a signature, SmartScreen warns the user (see the installation guide).

5. Android

```
QT_ANDROID=~/Qt/6.5.3/android_arm64_v8a QT_HOST=~/Qt/6.5.3/gcc_64 \
ANDROID_SDK_ROOT=~/Android/Sdk ANDROID_NDK_ROOT=~/Android/Sdk/ndk/25.2.9519653 \
scripts/build-android.sh --server-url https://api.example.cd
# -> dist/android/nhill-business-2.1.0-arm64-v8a.apk
```

QtKeychain (“Remember me”, Android Keystore) is required. Build it once from its verified source (tag 0.17.0, static library), then give its prefix:

```
QT_ANDROID=~/Qt/6.5.3/android_arm64_v8a packaging/android/build-qtkeychain-android.sh
~/build/qtkeychain-android
KEYCHAIN_PREFIX=~/build/qtkeychain-android QT_ANDROID=… QT_HOST=… ANDROID_SDK_ROOT=…
ANDROID_NDK_ROOT=… \
scripts/build-android.sh --server-url https://api.example.cd
```

Without `KEYCHAIN_PREFIX` the build stops; `ALLOW_NO_KEYCHAIN=1` produces an APK that never remembers the session. The script checks that QtKeychain’s Keystore code (`KeyGenParameterSpec`) is present in the application library. The real test (`.github/scripts/android-keychain-emulator.sh`) installs `test_keychain` on an Android 11 x86_64 emulator and checks writing, reading back after the application restarts, sign-out and erasure.

The SDK, NDK and Qt for Android do not come from `apt`: install them from their official sources (Google’s sdkmanager, Qt online installer) and check the published digests. Release signing: `KEYSTORE=… KEYSTORE_ALIAS=… KEYSTORE_PASS=…`; without these variables the APK is signed with a **debug** key (installable with `adb`, not publishable). The keystore must never be versioned; in CI it comes from the secrets `ANDROID_KEYSTORE_BASE64`, `ANDROID_KEYSTORE_ALIAS`, `ANDROID_KEYSTORE_PASS`.

6. Documentation PDFs

```
scripts/build-docs-pdf.sh            # regenerates every PDF listed in docs/pdf-manifest.txt
scripts/build-docs-pdf.sh --check    # fails if a PDF is missing or no longer matches its
Markdown
```

Markdown is the only source; `docs/pdf/manifest.json` records the digest of each source and each PDF. Never edit a PDF by hand.

7. Building with GitHub Actions

Platforms that cannot be built on your machine are built in CI:

1. Push the repository to a **private** GitHub repository.
2. Trigger the workflows: tag `git tag v2.1.0 && git push origin v2.1.0`, or “Run workflow” (Actions tab) for `linux-packages`, `windows`, `android`, `docs-website`.
3. Download the artefacts: `nhill-deb-ubuntu-24.04`, `nhill-deb-debian-13`, `nhill-windows-installers`, `nhill-android-apk`, `nhill-docs-and-website`.
4. Put them in `dist/linux/ubuntu-24.04/`, `dist/linux/debian-13/`, `dist/windows/`, `dist/android/`, `dist/website/`, then § 8.

With the `gh` tool: `gh run download <id> -n nhill-windows-installers -D dist/windows`.

8. Verify and release

```
scripts/verify-build.sh --require linux,docs        # check what exists
scripts/package-release.sh --force                  # assembles release/V2.1.0/
```

For every platform present, `verify-build.sh` checks the real file format (Debian package, PE executable, APK), the 2.x version, the `SHA256SUMS` sums, and the presence and freshness of the PDFs. `package-release.sh` copies only what exists into

`release/V2.1.0/{linux,windows,android,website,documentation,legal,marketing,source,checksums}` and lists what is missing in `checksums/MISSING.txt` — never a dummy file.

9. Reproducible builds: points of attention

- Build from a tagged, clean commit (empty `git status`); the source archive is produced by `git archive`.
- Record the tool versions (`deps-manifest-*.txt`, CI logs).
- Dates embedded in packages depend on the build clock; two builds are not necessarily byte-for-byte identical.